Gravitee Expression Language

Overview

Gravitee Expression Language (EL) queries and manipulates object graphs to dynamically configure API aspects and policies. Use EL to reference values from the current API transaction and create dynamic filters, routing rules, and policies that respond to specific conditions or parameters.

EL extends the Spring Expression Language (SpEL) by providing additional object properties inside the expression language context. The full SpEL syntax is available in EL. Gravitee has implemented customizations detailed in this guide.

For security, the EL engine restricts the expressions it evaluates: only the Java methods and constructors on its allowlist are callable, property access is read-only, and bean references aren't available. To make additional classes or methods callable, configure the `el.whitelist` section of the Gateway `gravitee.yml` file.

Object properties

Custom properties and attributes have special meanings in the Gravitee ecosystem:

- **Custom properties:** Defined at the API level and read-only during the Gateway's execution of an API transaction. Learn more about setting an API's custom properties in [link to custom properties documentation].
- **Attributes:** Scoped to the current API transaction and can be manipulated during the execution phase through the `assign-attributes` policy. Attributes attach additional information to a request or message via a variable that is dropped after the API transaction completes.

Basic usage

This section summarizes:

- Object properties added to the Expression Language (EL) context
- How attributes are accessed for v4 and v2 APIs
- Commonly used operators and functions

Syntax

Expressions

Expressions in Gravitee are enclosed in curly braces `{}` and begin with the `#` symbol. Both dot notation and bracket notation are supported for accessing object properties.

Example: `{#context.attributes['user'].email}`

Dot notation vs bracket notation

Dot notation doesn't work with special characters:

`{#request.headers.my-header}` ← **This will result in an error**

Use bracket notation for property names that include a space or hyphen, or start with a number:

`{#request.headers['my-header']}`

Lists

Expressions can be used to assign lists, for example: `{({'admin', 'writer'})}`

1. The outer curly braces start and end the EL expression
2. The parentheses indicate an object is being instantiated
3. The list comprises the inner brackets and enclosed values: `{'admin', 'writer'}`

Object properties

EL allows you to reference certain values injected into the EL context as object properties. The available object properties are detailed in later sections. EL adds the following root-level object properties:

- `{#api.properties}` : Custom properties defined by the API publisher for that Gateway API
- `{#dictionaries}` : Custom dictionaries defined by the API publisher for that Gateway API
- `{#endpoints}` : Information about the Gateway API's respective endpoints
- `{#request}` : Information about the current API request
- `{#response}` : Information about the current API response
- `{#message}` : Information about the current API message
- `{#node}` : Information about the node hosting the instance of the Gateway handling the API transaction
- `{#application}` : Information about the consumer's Application authenticated by the Gateway (for example: `{#application.metadata['some_key']}`)
- `{#subscription}` : Information about the consumer's Subscription authenticated by the Gateway (for example: `{#subscription.metadata['some_key']}`)

Attributes

The `attributes` object property contains attributes that are automatically created by the APIM Gateway during an API transaction or added during the execution phase through the Assign Attributes policy. Attributes fall into one of two categories based on API type:

- `{#context.attributes}`: Attributes associated with v2 APIs or v4 Proxy APIs. A v4 Proxy API is created using the **Proxy upstream protocol** method.
- `{#message.attributes}`: Attributes associated with v4 Message APIs. These APIs are created using the **Introspect messages from event-driven backend** method.

See the v4 API creation wizard for more details.

Operators

EL supports various operators, such as arithmetic, logical, comparison, and ternary operators. Examples of commonly used operators in Gravitee include:

- Arithmetic operators: `+`, `-`, `*`, `/`
- Logical operators: `&&` (logical and), `||` (logical or), `!` (logical not)
- Comparison operators: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Ternary operators: `condition ? expression1 : expression2`

Functions

EL provides a variety of built-in functions to manipulate and transform data in expressions. Examples of commonly used functions in Gravitee include:

- String functions: `length()`, `substring()`, `replace()`

- `#jsonPath`: Evaluates a JSONPath expression on a specified object. This function delegates to the Jayway JSONPath library. The best way to learn JSONPath syntax is by using the online evaluator.
- `#xpath`: Evaluates an `xpath` on a provided object. For more information regarding XML and XPath, see XML Support - Dealing with XML Payloads in the SpEL documentation.

`jsonPath` example

As an example of how `jsonPath` can be used with EL, suppose you have a JSON payload in the request body that contains the following data:

```
{
  "store": {
    "book": [
      {
        "category": "fiction",
        "author": "Herman Melville",
        "title": "Moby Dick",
        "isbn": "0-553-21311-3",
        "price": 8.99
      },
      {
        "category": "fiction",
        "author": "J. R. R. Tolkien",
        "title": "The Lord of the Rings",
        "isbn": "0-395-19395-8",
        "price": 22.99
      }
    ]
  }
}
```

To extract the value of the `price` property for the book with `title` "The Lord of the Rings," use the following expression:

`{#jsonPath(#request.content, "$.store.book[?(@.title=='The Lord of the Rings')].price")}`

Request/Response body access

You can access the request/response raw content using `{#request.content}`.

Depending on the content-type, you can access specific content.

JSON content

> If a JSON payload has duplicate keys, APIM keeps the last key.
>
> To avoid errors caused by duplicate keys, apply the JSON threat protection policy to the API. For more information about the JSON threat protection policy, see https://github.com/gravitee-io/gravitee-platform-docs/tree/main/docs/apim/4.12/create-and-configure-apis/apply-policies/policy-reference/json-threat-protection/README.md.

You can access specific attributes of a JSON request/response payload with `{#request.jsonContent.foo.bar}`, where the request body is similar to the following example:

```
{
  "foo": {
      "bar": "something"
  }
}
```

XML content

You can access specific tags of an XML request/response payload with `{#request.xmlContent.foo.bar}`, where the request body is similar to the following example:

```
<foo>
  <bar>something</bar>
</foo>
```

EL syntax and evaluation rules

Gravitee EL wraps Spring Expression Language (SpEL) with a template parser that recognizes three expression markers inside strings. Everything outside a marker is treated as literal text.

Expression markers

The EL template parser recognizes three markers, with optional whitespace allowed after the opening `{`:

- `{#...}`: evaluates a SpEL expression. This is the canonical EL marker.
- `{(...)}`: evaluates a parenthesized SpEL expression. Useful for inline evaluation inside a larger string.
- `{T(...)}`: evaluates a SpEL type reference, for example `{T(java.lang.String).format(...)}`.

Any `{...}` block that doesn't start with `#`, `(`, or `T` is left untouched and returned as literal text.

Condition fields

Gateway condition fields (for example, policy conditions and flow conditions) are evaluated by the gateway as a Boolean. The Gateway passes the raw field value to the template engine with `Boolean.class` as the target type. The template engine returns a Boolean, which the gateway uses to decide whether the condition matches.

If evaluation throws an `ExpressionEvaluationException`, the gateway logs a warning, raises an `EXPRESSION_EVALUATION_ERROR` execution warning, and treats the condition as non-matching (the element is filtered out).

Verified evaluation examples

The following examples reflect the behavior shipped with the Gateway:

Input	Target type	Result
`true`	`Boolean`	`true`
`{#request.headers['X-Gravitee-Endpoint'] == null}`	`Boolean`	Boolean result of the comparison
`{#request.content.startsWith('pong')}`	`Boolean`	Boolean result of the comparison
`{1 == 1}`	`String`	Literal `{1 == 1}` (not evaluated, no EL marker)
`{(1 == 1)}`	`String`	`true`
`{(12 == 1)}`	`String`	`false`

> The EL template parser only treats a `{...}` block as an expression if the first non-whitespace character inside is #, (, or T. A bare input like 1==1 or true written without any marker is returned as literal text by the template engine. For a Boolean condition field, wrap the comparison in an EL marker, for example `{#request.headers['X-Debug'] != null}` so the gateway evaluates it.

EL versus Apache FreeMarker

Gravitee uses two different template languages in different places. They aren't interchangeable.

Where EL is used

EL is used in Gateway execution paths, including:

- Policy and flow condition fields.
- Any field documented as supporting EL.

EL expressions use `{#...}`, `{(...)}`, or `{T(...)}` markers.

Where FreeMarker is used

Gravitee notifiers render their configuration payloads (for example, the webhook notifier body) through Apache FreeMarker. The notifier base class builds a FreeMarker `Template` from the configured payload and processes it against a parameters map before dispatching the notification.

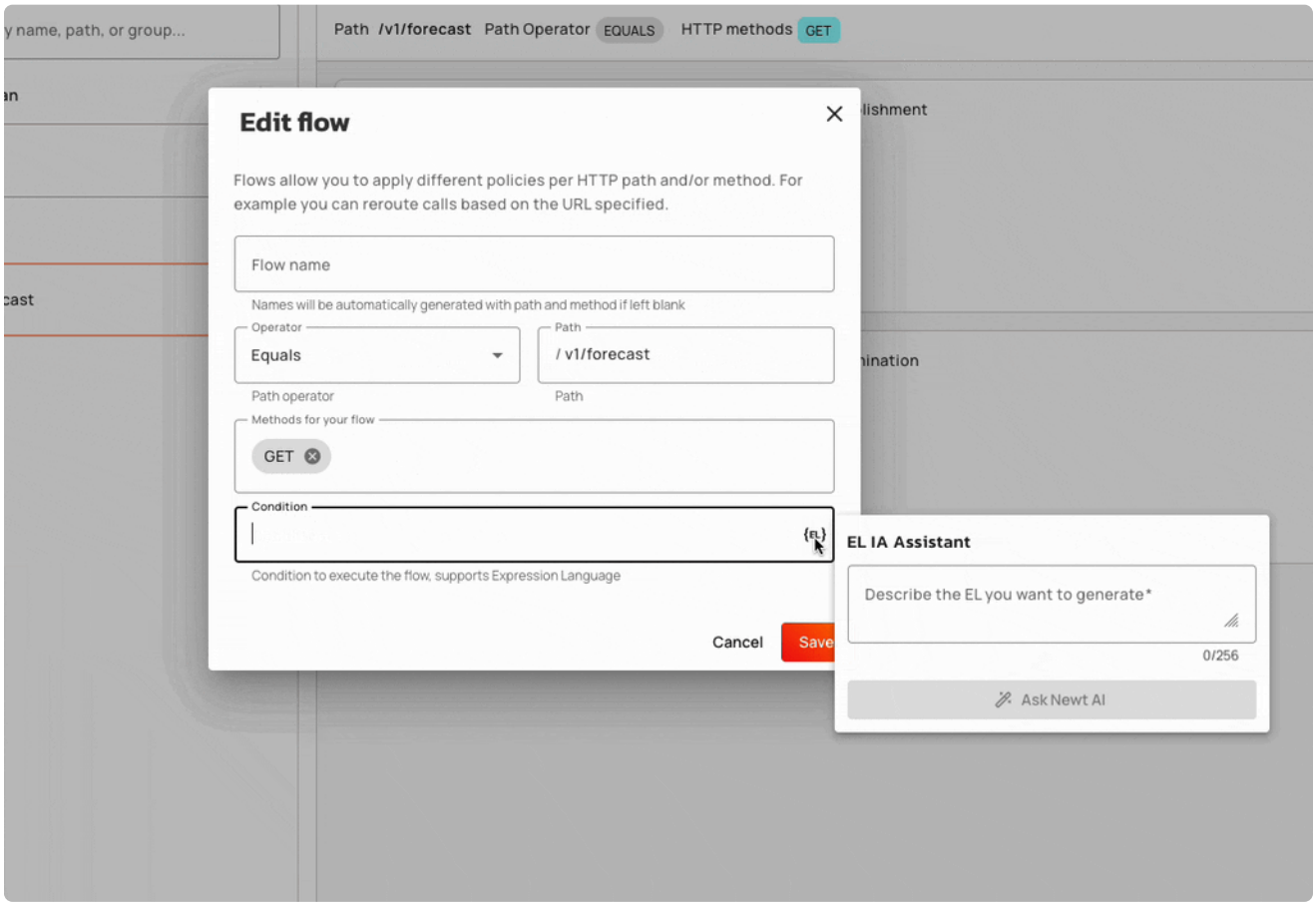
FreeMarker uses `${...}` placeholders and its own directive syntax, which is different from EL. For FreeMarker syntax, see the Apache FreeMarker documentation ↗.

The notifier body is processed by FreeMarker, not the EL template engine. The EL parser doesn't recognize `${...}` as an expression marker, so FreeMarker syntax in a Gateway condition field is treated as literal text. Use the language that matches the field.

Expression Language Assistant

Overview

The Expression Language (EL) Assistant generates EL expressions based on natural language prompts. You describe the condition or logic you need, and the Assistant returns the corresponding EL syntax.



Prerequisites

Before using the EL Assistant, complete the following:

- Register for a Gravitee Cloud account at Cloud ↗.
- (Self-hosted and Hybrid installations only) Register your installation in Gravitee Cloud. For more information, see Register installations. ↗
- (Self-hosted and Hybrid installations only) Enable the EL Assistant by adding the following configuration:
- Enable the EL Assistant on the Management API.

> The `newtai.elgen.enabled` setting is read by the Management API, not the Gateway. Apply it to the Management API's `gravitee.yml`, the Management API container's environment, or the `api.newtai.elgen` block of `values.yaml`. Setting it on the Gateway has no effect.

Gravitee Cloud (SaaS) deployments

In a Gravitee Cloud deployment, you don't operate the Management API, so you can't edit `gravitee.yml` or `values.yaml` yourself. Contact Gravitee support ↗to request that the EL Assistant be enabled on your environment.

Self-hosted and Hybrid deployments

Apply one of the following configurations to the Management API.

gravitee.yml

Add the following configuration at the root level of the Management API's `gravitee.yml` file:

```yaml
newtai:
  elgen:
    enabled: true
```

.env

Add the following line to the `.env` file loaded by the Management API container, or to that container's `environment:` block:

```
gravitee_newtai_elgen_enabled=true
```

Helm values.yaml

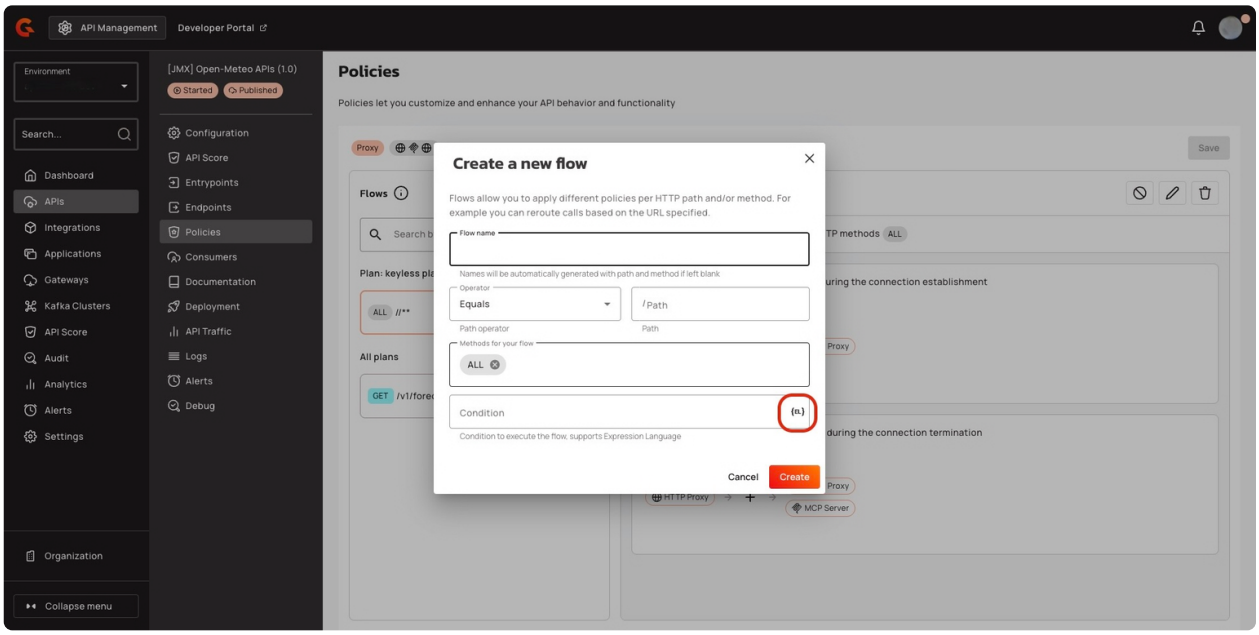
Add the following configuration under the `api:` block of your `values.yaml` file. The key nests under `api.newtai.elgen`, not at the root of `values.yaml`.

```yaml
api:
  newtai:
    elgen:
      enabled: true
```

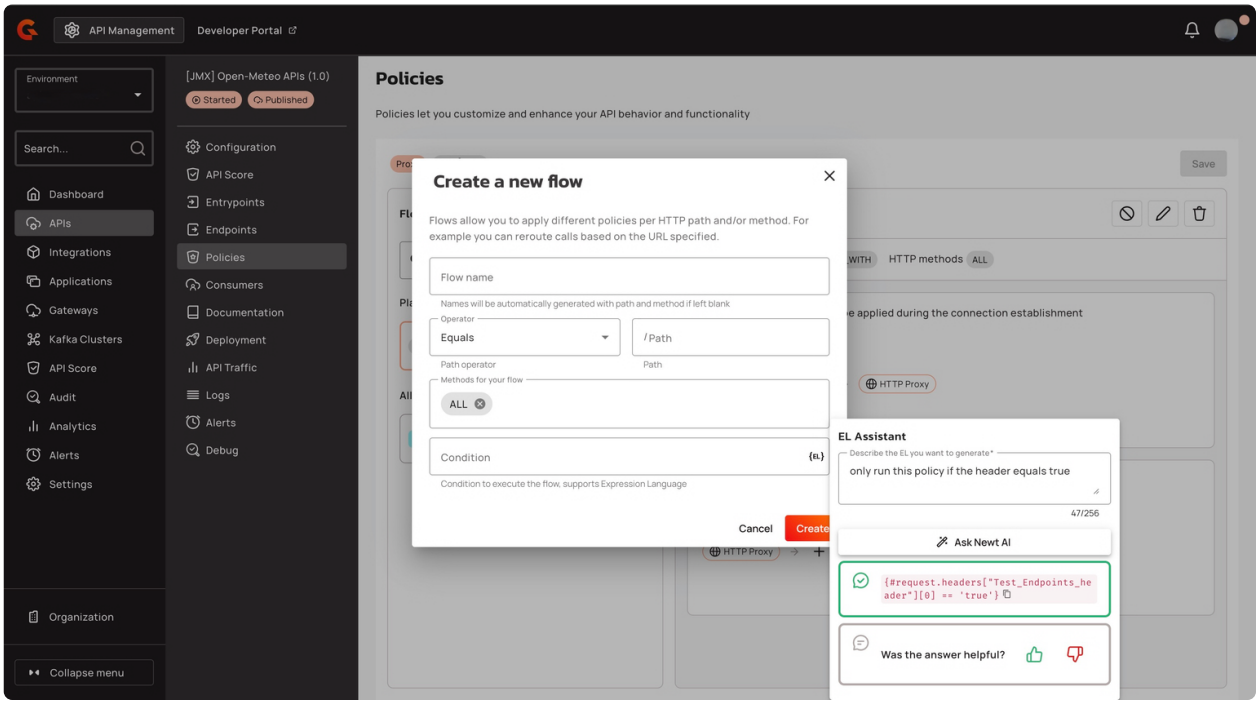
Generate Expression Language with the EL Assistant

The EL Assistant is available in any field that supports Expression Language.

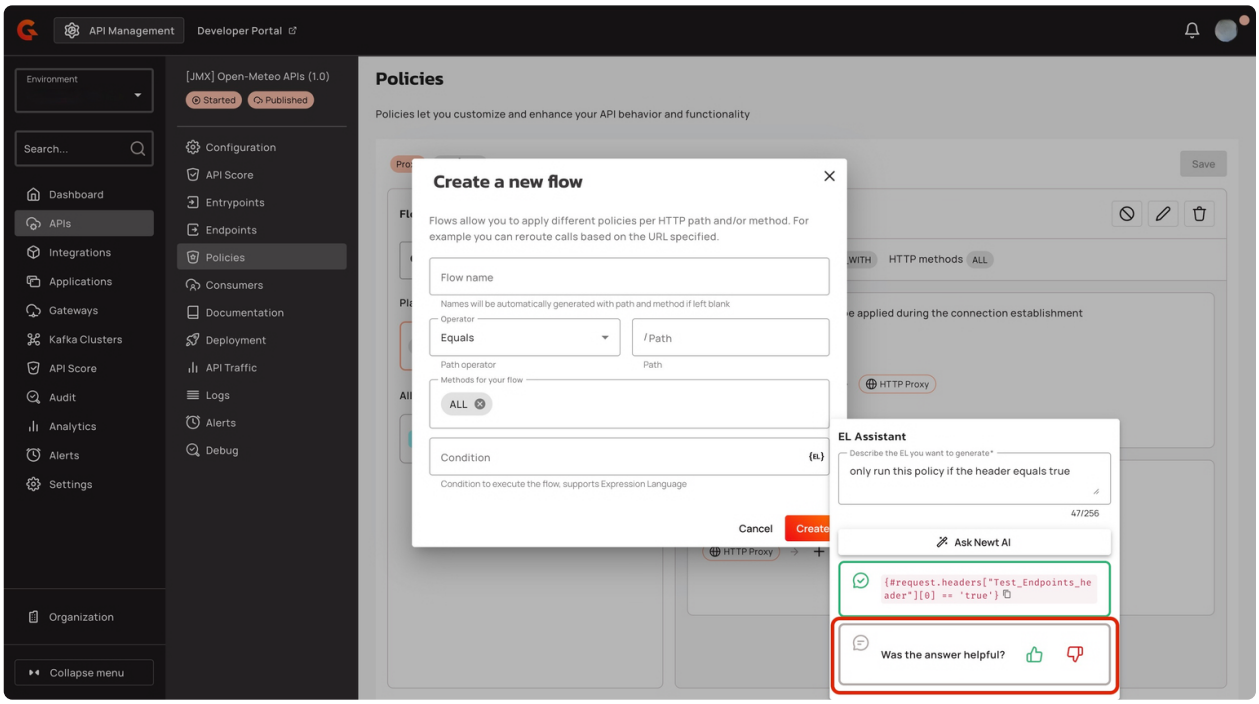
1. In the field that supports Expression Language, click the **{EL}** icon.



2. In the **EL Assistant** pop-up window, enter a natural language prompt describing the Expression Language you need. For example: "Only run this policy if the header equals test."
3. Click **Ask Newt AI**. The Assistant generates the corresponding Expression Language.



4. (Optional) Provide feedback by clicking the **thumbs up** or **thumbs down** icon.



Use case examples

The following examples demonstrate how to use the EL Assistant for common tasks.

Add a condition to a policy flow

To run a policy flow only when a header equals "test," enter the following prompt:

"Only run this policy flow if the header equals test."

The EL Assistant returns:

```
{#request.headers['Test_Endpoints_header'][0] == 'test'}
```

Target an endpoint

To configure a target URL for an endpoint that points to `https://jsonplaceholder.typicode.com/`, enter the following prompt:

"The target URL must target the following URL: https://jsonplaceholder.typicode.com/."

The EL Assistant returns:

```
{#context.attributes['https://jsonplaceholder.typicode.com']}
```

Add an assertion for API health checks

To create an assertion that validates an HTTP 200 response, enter the following prompt:

"Check only the status of the following HTTP response that equals 200."

The EL Assistant returns:

```
{#response.status == 200}
```

APIs

Use the Gravitee Expression Language (EL) to access API transaction information through root-level objects injected into the EL context: custom properties, dictionaries, and endpoints.

API object properties

The `{#api}` root-level object exposes metadata about the Gateway API handling the current transaction.

Object property	Description	Type	Example
id	Gateway API identifier	string	`{#api.id}`
name	Gateway API name	string	`{#api.name}`
version	Gateway API version, as declared on the API definition	string	`{#api.version}`
properties	Custom properties defined on the API. See the Custom properties tab below for details	key / value	`{#api.properties['my-property']}`

Custom properties

Define custom properties for your API. These properties are automatically injected into the expression language context and can be referenced during an API transaction from the `{#api.properties}` root-level object property.

Examples

- Get the value of the property `my-property` defined in an API's custom properties: `{#api.properties['my-property']}`
- Get the value of the property `my-secret` defined and encrypted in an API's custom properties: `{#api.properties['my-secret']}`

> **Encrypted custom properties**
>
> When you access an encrypted custom property, the Gateway automatically decrypts it and provides a plain text value.

Dictionaries

Dictionaries work similarly to custom properties, but you must specify both the dictionary ID and the dictionary property name. Dictionary properties are key-value pairs that can be accessed from the `{#dictionaries}` root-level object property.

Example

Get the value of the dictionary property `dict-key` defined in dictionary `my-dictionary-id` : `{#dictionaries['my-dictionary-id']['dict-key']}` .

Endpoints

When you define endpoints for your API, assign each a unique name across all endpoints. Use this identifier to get an endpoint reference (a URI) from the `{#endpoints}` root-level object property.

Example

When you create an API, a default endpoint is created that corresponds to the backend property value. Retrieve this endpoint using the following syntax: `{#endpoints['default']}`.

Request

EL can be used to access request properties and attributes as described below.

Request object properties

The object properties you can access from the `{#request}` root-level object property and use for API requests are listed below.

Table

Object property	Description	Type	Example
content	Body content	string	-
contextPath	Context path	string	/v2/
headers	Headers	key / value	X-Custom: myvalue

host	The host of the request. This is preferable to using the Host header of the request because HTTP2 requests do not provide this header.	string	gravitee.example.com
id	Identifier	string	12345678-90ab-cdef-1234-567890ab
localAddress	Local address	string	0:0:0:0:0:0:0:1
method	HTTP (Hypertext Transfer Protocol) method	string	GET
params	Query parameters	key / value	order: 100
path	Path	string	/v2/store/MyStore
pathInfo	Path info	string	/store/MyStore
pathInfos	Path info parts	array of strings	[,store,MyStore]
pathParams	Path parameters	key / value	storeId: MyStore (*see Warning for details*)
pathParamsRaw	Path parameters	string	/something/:id/**
paths	Path parts	array of strings	[,v2,store,MyStore]
remoteAddress	Remote address	string	0:0:0:0:0:0:0:1
scheme	The scheme of the request (either `http` or `https`)	string	http
	SSL (Secure		

ssl	SSL (Secure Sockets Layer) session information	SSL object	-
timestamp	Timestamp	long	1602781000267
transactionId	Transaction identifier	string	cd123456-7890-abcd-ef12-34567890
uri	URI (Uniform Resource Identifier)	string	/v2/store/MyStore?order=100
version	HTTP version	string	HTTP_1_1

Examples

- Get the value of the `Content-Type` header for an incoming HTTP request: `{#request.headers['content-type']}`
- Get the second part of the request path: `{#request.paths[1]}`

Request context attributes

When APIM Gateway handles an incoming API request, some object properties are automatically created or added during the execution phase through the Assign Attributes policy. These object properties are known as attributes. Attributes can be accessed from the `{#context.attributes}` root-level object property.

Some policies (e.g., the OAuth2 policy) register other attributes in the request context. For more information, refer to the documentation for individual policies.

Request context attributes and examples are listed below:

Table

Attribute key	Description	Type	Set by
api	ID of the API handling the request	string	Gateway (always)
api.name	Name of the API handling the request	string	Gateway (always)
api.deployed-at	Timestamp of the last API deployment	long	Gateway (always)
api-key	The API key used by the consumer	string	API Key policy (only on API Key plans)
application	ID of the authenticated application	string	Security chain (empty for Keyless plans)
apiProduct	ID of the API Product associated with the current subscription	string	Subscription processor (only when the subscription belongs to an API Product)
plan	ID of the plan the consumer is subscribed to	string	Security chain
user	Authenticated user identifier	string	Security chain
user.roles	Roles granted to the authenticated	list / array	Security chain

	user		
user-id	The user identifier of the incoming HTTP request. The subscription ID for an API Key plan, the remote IP for a Keyless plan	string	Gateway (always)
clientIdentifier	Stable client identifier used for rate limiting and logging. Derived from the subscription ID, a header, or the remote address	string	Subscription processor (always)
organization	ID of the organization the API belongs to	string	Gateway (always)
environment	ID of the environment the API is deployed in	string	Gateway (always)
context-path	Context path configured on the API listener	string	Gateway (always)
resolved-path	Path resolved from the flow selectors: the path defined in policies	string	Flow chain
mapped-path	Path mapping matched for analytics and logging	string	Gateway (always)
	HTTP method of		

request.method	HTTP method of the current request	string	Gateway (always)
request.endpoint	Endpoint URI selected for the current request	string	Gateway (always)
request.endpoint.override	Endpoint URI explicitly overridden for the current request (for example, by the Dynamic Routing policy)	string	Dynamic routing / assign-attributes (only when overridden)
request.original-url	Full URL of the original request received by the Gateway, including scheme, host, and path	string	Gateway (always)
quota.count	Current value of the quota counter for the active subscription	long	Quota policy (only when a Quota policy is applied)
quota.limit	Configured quota limit	long	Quota policy (only when a Quota policy is applied)
quota.remaining	Remaining quota for the active subscription	long	Quota policy (only when a Quota policy is applied)
quota.reset.time	Timestamp at which the quota counter resets	long	Quota policy (only when a Quota policy is applied)
sni	Server Name Indication value from the TLS	string	Gateway (only when TLS SNI is

	handshake		used)

Examples

- Get the value of the `user-id` attribute for an incoming HTTP request: `{#context.attributes['user-id']}`
- Get the value of the `plan` attribute for an incoming HTTP request: `{#context.attributes['plan']}`
- Get the authenticated application ID: `{#context.attributes['application']}`
- Get the stable client identifier: `{#context.attributes['clientIdentifier']}`

Attribute keys without the `gravitee.attribute.` prefix

The Gateway stores context attributes with a `gravitee.attribute.` prefix (for example, `gravitee.attribute.plan`). Expression Language lookups through `{#context.attributes['...']}` accept either the short key or the fully prefixed key. Use the short keys shown in the table above for readability.

SSL object properties

The object properties you can access in the `ssl` session object from the `{#request.ssl}` root-level object property are listed below.

Table

Object property	Description	Type	Example
clientHost	Host name of the client	string	client.domain.com
clientPort	Port number of the client	long	443
client	Client information	Principal object	-
server	Server information	Principal object	-

Example

Get the client HOST from the SSL session: `{#request.ssl.clientHost}`

Principal objects

The `client` and `server` objects are of type `Principal`. A `Principal` object represents the currently authenticated user who is making the request to the API and provides access to various user attributes such as username, email address, roles, and permissions.

The `Principal` object is typically used with security policies such as OAuth2, JWT (JSON Web Token), or basic authentication to enforce access control and authorization rules on incoming requests. For example, a policy can check if the current user has a specific role or permission before allowing them to access a protected resource.

If the `Principal` object is not defined, `client` and `server` object values are empty. Otherwise, there are domain name attributes you can access from the `{#request.ssl.client}` and `{#request.ssl.server}` `Principal` objects as shown in the table below:

> **Limitation on arrays**
>
> All attributes of the `Principal` object are flattened to be accessed directly with dot or bracket notation. While some of these attributes can be arrays, EL will only return the first item in the array. To retrieve all values of an attribute, use the `attributes` object property shown in the table and examples below.

Table

Object property	Description	Type	Example
attributes	Retrieves all the `Principal` object's domain name attributes	key / value	"ou": ["Test team", "Dev team"]
businessCategory	Business category	string	-
c	Country code	string	FR
cn	Common name	string	-
countryOfCitizenship	RFC 3039 CountryOfCitizenship	string	-
countryOfResidence	RFC 3039 CountryOfResidence	string	-
dateOfBirth	RFC 3039 DateOfBirth	string	198307190000000Z

dc	Domain component	string	-
defined	Returns `true` if the `Principal` object is defined and contains values. Returns `false` otherwise.	boolean	-
description	Description	string	-
dmdName	RFC 2256 directory management domain	string	-
dn	Fully qualified domain name	string	-
dnQualifier	Domain name qualifier	string	-
e	Email address in Verisign certificates	string	-
emailAddress	Email address (RSA PKCS#9 extension)	string	-
gender	RFC 3039 Gender	string	"M", "F", "m" or "f"
generation	Naming attributes of type X520name	string	-
givenname	Naming attributes of type X520name	string	-
initials	Naming attributes of type X520name	string	-
l	Locality name	string	-
name	Name	string	-

nameAtBirth	ISIS-MTT NameAtBirth	string	-
o	Organization	string	-
organizationIdentifier	Organization identifier	string	-
ou	Organization unit name	string	-
placeOfBirth	RFC 3039 PlaceOfBirth	string	-
postalAddress	RFC 3039 PostalAddress	string	-
postalCode	Postal code	string	-
pseudonym	RFC 3039 Pseudonym	string	-
role	Role	string	-
serialnumber	Device serial number name	string	-
st	State or province name	string	-
street	Street	string	-
surname	Naming attributes of type X520name	string	-
t	Title	string	-
telephoneNumber	Telephone number	string	-
uid	LDAP (Lightweight Directory Access	string	-

	Protocol) User id		
uniqueIdentifier	Naming attributes of type X520name	string	-
unstructuredAddress	Unstructured address (from PKCS#9)	string	-

Examples

Standard object properties

- Get the client DN from the SSL session: `{#request.ssl.client.dn}`
- Get the server organization from the SSL session: `{#request.ssl.server.o}`

Arrays and boolean logic

- Get all the organization units of the server from the SSL session:
  - `{#request.ssl.server.attributes['ou'][0]}`
  - `{#request.ssl.server.attributes['OU'][1]}`
  - `{#request.ssl.server.attributes['Ou'][2]}`
- Get a custom attribute of the client from the SSL session: `{#request.ssl.client.attributes['1.2.3.4'][0]}`
- Determine if the SSL attributes of the client are set: `{#request.ssl.client.defined}`

Response

The `{#response}` root-level object property provides access to API response data. The following table lists the available object properties.

Table

Object property	Description	Type	Example
content	Body content	string	-
headers	Headers	key / value	X-Custom: myvalue
status	Status of the HTTP response	int	200

Example

Get the status of an HTTP response:

```
{#response.status}
```

Message

The `{#message}` root-level object property provides access to API message properties. A message (sent or received) may contain attributes that can be retrieved via `{#message.attributes[key]}`.

> The EL (Expression Language) used for a message does not change based on phase. EL is executed on the message itself, so whether the message is sent in the subscribe or publish phase is irrelevant.

Table

Object property	Description	Type	Example
attributeNames	The names of the attributes	list / array	-
attributes	Attributes attached to the message	key / value	-
content	Content of the message	string	-
contentLength	Size of the content	integer	-
error	Flag regarding the error state of the message	boolean	-
headers	Headers attached to the message	key / value	-
id	ID of the message	string	-
metadata	Metadata attached to the message	key / value	-

Examples

- Get the value of the `Content-Type` header for a message: `{#message.headers['content-type']}`
- Get the size of a message: `{#message.contentLength}`

Writable message attributes by endpoint type

On v4 Message APIs, some endpoint connectors read writable attributes off the current context or message and use them to override their runtime behavior. Set these keys with the Assign Attributes policy to change connector behavior per request or per message.

> The keys in this section apply only to v4 Message APIs that attach the Kafka **endpoint connector**. These are APIs built with the **Introspect messages from event-driven backend** method (protocol mediation). For Kafka APIs built on the **Kafka Gateway** (native Kafka protocol), these keys have no effect: the Kafka Gateway routes traffic at the Kafka protocol level and doesn't read these Gravitee attributes. See the Kafka Gateway section below for the EL surface available on Kafka APIs.

Kafka endpoint

The Kafka endpoint connector reads the following writable attributes. There are two prefixes: `gravitee.attribute.kafka.*` (hand-coded attribute names the connector reads at specific points) and `gravitee.attributes.endpoint.kafka.*` (handled by the endpoint's configuration evaluator and able to override any field on the shared configuration).

Attribute key	Reads from	Effect
`gravitee.attribute.kafka.topics`	Context (consumer and producer), message (producer fallback)	Overrides the consumer topic list, or the producer topic list when no `producer.topics` attribute is set on the message.
`gravitee.attribute.kafka.groupId`	Context	Overrides the consumer group ID for the request.
`gravitee.attribute.kafka.recordKey`	Message, falls back to context	Sets the produced record's key.

`gravitee.attribute.kafka.avroKey`	Message	Sets a raw byte-buffer key on the produced record. Short-circuits `recordKey` when set, and adds an `avroKey` header to the record.
`gravitee.attributes.endpoint.kafka.producer.topics`	Message	Per-message override of the producer topic list. Read before the `kafka.topics` fallback. Routes the message to a different topic than the one configured on the endpoint.
`gravitee.attributes.endpoint.kafka.<field>`	Context	Per-request override of any field on the endpoint's shared configuration. For example, `gravitee.attributes.endpoint.kafka.consumer.topics` overrides the consumer topic list, and `gravitee.attributes.endpoint.kafka.security.sasl.saslMechanism` overrides the SASL mechanism.

For full details on what each Kafka attribute does and when the connector reads it, see the Kafka endpoint reference.

Message metadata by endpoint type

On v4 Message APIs (**Introspect messages from event-driven backend**), the gateway populates `{#message.metadata}` with keys specific to the endpoint connector that consumed the message from the backend. Use these keys in policy conditions, assign-attributes expressions, and dynamic routing to react to protocol-level data without parsing the payload.

> The **Kafka Gateway** section below documents a separate `{#message}` surface for Kafka-native APIs built on the Kafka Gateway. The metadata keys listed here apply to v4 Message APIs attached to the Kafka endpoint connector, not to the Kafka Gateway.

Kafka

The Kafka endpoint populates the following metadata keys on each consumed record.

Metadata key	Description	Type	Example
topic	Name of the Kafka topic the record was consumed from	string	`{#message.metadata['topic']}`
partition	Partition number the record was consumed from	int	`{#message.metadata['partition']}`
offset	Offset of the record within its partition	long	`{#message.metadata['offset']}`
key	Record key. Only set when the record has a key. Byte-array keys are decoded as UTF-8 strings.	string	`{#message.metadata['key']}`

Examples

- Route only messages from the `orders` topic:

```
{#message.metadata['topic'] == 'orders'}
```

- Apply a policy only to records from partition `0`:

```
{#message.metadata['partition'] == 0}
```

- Build a CloudEvents `source` from the record coordinates:

```
kafka://{#message.metadata['topic']}/{#message.metadata['partition']}/{#message.metadata['offset']}
```

- Forward the record key as an upstream header value:

```
{#message.metadata['key']}
```

MQTT5

The MQTT5 endpoint populates the following metadata keys on each received publish packet.

Metadata key	Description	Type	Example
topic	Topic the packet was published on	string	`{#message.metadata['topic']}`
type	MQTT5 packet type name reported by the client library	string	`PUBLISH`
qos	Numeric QoS level (`0`, `1`, or `2`)	int	`{#message.metadata['qos']}`
retain	Whether the retain flag was set on the publish	boolean	`{#message.metadata['retain']}`
contentType	MQTT5 content-type property, when the publisher set it	string	`{#message.metadata['contentType']}`
messageExpiryInterval	MQTT5 message expiry interval, in seconds. Returns `-1` when the publisher didn't set it.	long	`{#message.metadata['messageExpiryInterval']}`
responseTopic	MQTT5 response-topic property, when the publisher set it	string	`{#message.metadata['responseTopic']}`

Examples

- Run a policy only for QoS 2 messages:

```
{#message.metadata['qos'] == 2}
```

- Route retained messages to a dedicated flow:

```
{#message.metadata['retain'] == true}
```

Solace

The Solace endpoint populates the following metadata keys on each received message.

Metadata key	Description	Type	Example
senderId	Sender ID reported by Solace for the message	string	`{#message.metadata['senderId']}`
destinationName	Queue or topic name the message was consumed from	string	`{#message.metadata['destinationName']}`
senderTimestamp	Sender-side timestamp in epoch milliseconds. Returns `-1` when the message doesn't carry one.	long	`{#message.metadata['senderTimestamp']}`
isCached	Whether the message was delivered from the Solace cache	boolean	`{#message.metadata['isCached']}`
isRedelivered	Whether the message is a redelivery	boolean	`{#message.metadata['isRedelivered']}`

Example

- Skip redelivered messages in a filtering policy:

```
{#message.metadata['isRedelivered'] == false}
```

RabbitMQ

The RabbitMQ endpoint always populates three envelope keys on every delivery. The remaining keys appear only when the publisher set the corresponding AMQP property.

Metadata key	Description	Type	Set when
deliveryTag	Broker-assigned delivery tag for the current delivery	long	Always
exchange	Exchange the message was published through	string	Always
routingKey	Routing key used when the message was published	string	Always
appId	AMQP `app-id` property	string	Set by publisher
expiration	AMQP `expiration` property, as a string in milliseconds	string	Set by publisher
type	AMQP `type` property	string	Set by publisher

deliveryMode	AMQP `delivery-mode` (`1` for non-persistent, `2` for persistent)	int	Set by publisher
replyTo	AMQP `reply-to` property	string	Set by publisher
priority	AMQP `priority` property	int	Set by publisher
timestamp	AMQP `timestamp` property	date	Set by publisher
userId	AMQP `user-id` property	string	Set by publisher

Example

- Route messages by routing key prefix:

```
{#message.metadata['routingKey'].startsWith('payments.')}
```

Kafka Gateway

The Kafka Gateway uses a distinct set of Expression Language root objects that reflect Kafka protocol concepts rather than HTTP concepts. Use this section when configuring EL on Kafka APIs, for example, in endpoint configuration, policy conditions, or connection interruption rules.

The `#request`, `#response`, and `#message` objects on Kafka APIs expose different properties than the HTTP versions. HTTP-specific properties such as `#request.method`, `#request.headers`, or `#request.path` aren't available on Kafka APIs.

EL availability by phase

The set of root objects available to an EL expression depends on the execution phase. Kafka APIs run through four distinct phases, each exposing a different variable surface.

Phase	When it runs	Available root objects
Entrypoint Connect	Before the Kafka client authenticates	`#connection`, `#ssl`, `#context` (attributes and addresses only, no `principal`)
Connection	After the client authenticates, once per connection	`#context` (with `principal` and `ssl`)
Request	Per Kafka protocol request (Produce, Fetch, Metadata, and others)	`#request`, `#response`, `#context`
Message	Per Kafka record, for message-level policies	`#message`, plus everything available in the Request phase

Kafka context object properties

The `#context` root object on a Kafka API exposes connection-level and authentication data for the current client connection.

Object property	Description	Type	Example
attributes	Context attributes associated with the Kafka connection	key / value	`{#context.attributes['plan']}`
remoteAddress	Remote address of the Kafka client	string	`{#context.remoteAddress}`
localAddress	Local address of the gateway listener	string	`{#context.localAddress}`
ssl	TLS session information for the client connection	SSL object	`{#context.ssl.clientHost}`
principal	Authenticated Kafka principal for the current connection	Principal object	`{#context.principal.name}`

Kafka principal object properties

The `#context.principal` object exposes the authenticated Kafka client identity.

Object property	Description	Type	Nullable
name	The principal name as seen by the Kafka broker (for example, the SASL username or the certificate common name)	string	Yes. Returns `null` when no principal is set
token	The raw bearer token presented by the client	string	Yes. Populated only when the client authenticates with SASL OAUTHBEARER. Returns `null` for all other authentication methods, including PLAINTEXT, mTLS, SASL PLAIN, SASL SCRAM, and API Key plans

Pass the client's OAuth token to the broker

When a Kafka API is secured with SASL OAUTHBEARER and the gateway forwards the client's bearer token to the upstream Kafka broker, configure the endpoint's bearer token field as follows:

```
{#context.principal.token}
```

The gateway extracts the token from the authenticated client connection and reuses it when authenticating to the broker.

Kafka request object properties

The `#request` root object on a Kafka API exposes Kafka protocol-level metadata for the current operation. It's only available in the Request phase.

Object property	Description	Type	Example
correlationId	Correlation ID of the Kafka request, returned as a string	string	`12345`
apiKey	Name of the Kafka protocol operation	string	`PRODUCE`, `FETCH`, `METADATA`
apiVersion	Version of the Kafka protocol operation	int	`10`
clientId	Client ID sent by the Kafka client in the request header	string	`my-kafka-producer`

> **`#request.apiKey` isn't the Gravitee API Key plan.** On Kafka APIs, `#request.apiKey` returns the Kafka protocol operation name (for example, `PRODUCE` or `FETCH`). To read the API key value used by a Gravitee API Key plan, use `{#context.attributes['api-key']}` instead.

Kafka response object properties

The `#response` root object is registered for Kafka APIs but doesn't currently expose any properties. Don't rely on `#response.*` expressions on Kafka APIs.

Kafka message object properties

For policies that run on individual Kafka records, for example the Kafka Message Filtering policy, the `#message` root object exposes record-level data.

Object property	Description	Type	Example
topic	Name of the Kafka topic the record belongs to	string	`orders`
key	Record key, as a string	string	`customer-42`
content	Record value body, as a string	string	`{"total":100}`
contentLength	Length of the record value body	int	`14`
headersString	Record headers with values decoded as strings	key / value	`{#message.headersString['trace-id']}`
headersRaw	Record headers with raw byte-array values	key / value	-
metadata	Metadata attached to the message by the gateway	key / value	`{#message.metadata['partition']}`
error	Flag indicating whether the record is in an error state	boolean	`false`

Entrypoint Connect phase context

At the Entrypoint Connect phase, where policies run before the Kafka client authenticates, such as connection interruption rules, the EL context is intentionally reduced. Only connection-level data is available.

Root object	Property	Description
`#connection`	id	Stable identifier for the client connection
`#connection`	remoteAddress	Address of the Kafka client
`#connection`	localAddress	Address of the gateway listener
`#ssl`	(SSL object)	TLS session for the client connection, if SSL is configured
`#context`	attributes	Context attributes set before authentication
`#context`	remoteAddress	Remote address of the Kafka client
`#context`	localAddress	Local address of the gateway listener

`#context.principal`, `#request`, `#response`, and `#message` aren't available in the Entrypoint Connect phase because authentication hasn't occurred and no Kafka protocol request has been received yet. Referencing them in an EL expression evaluated at this phase returns `null` or raises an evaluation error.

Subscription

The `{#subscription}` root-level object exposes information about the consumer subscription that authenticated the current API transaction. It's available once the security chain has resolved a subscription in practice, from the Request phase onward for every non-Keyless plan.

Object property	Description	Type	Example
id	Subscription identifier	string	`{#subscription.id}`
type	Subscription type. One of `STANDARD` or `PUSH`	string	`{#subscription.type}`
applicationName	Name of the application that owns the subscription	string	`{#subscription.applicationName}`
clientId	OAuth2 client ID associated with the subscription, if the plan uses one	string	`{#subscription.clientId}`
apiProductId	Identifier of the API Product associated with the subscription, if the subscription was created through an API Product	string	`{#subscription.apiProductId}`
metadata	Key / value metadata attached to the subscription by the API publisher	key / value	`{#subscription.metadata['clientType']}`

Referencing the authenticated application from EL

To reference the application that owns the current subscription, use `{#subscription.applicationName}` for the application name or `{#context.attributes['application']}` for the application ID. There's no dedicated `#application` root-level object.

Examples

- Route based on the subscribing application: `{#subscription.applicationName == 'partner-portal'}`
- Compare a subscription metadata value to a literal string: `{#subscription.metadata['clientType'].equals('PARTNER')}`
- Forward the subscription ID to the upstream as a header: `{#subscription.id}`

Nodes

A node is a component that represents an instance of the Gravitee Gateway. Each node runs a copy of the Gateway that handles incoming requests, executes policies, and forwards requests to upstream services. The object properties you can access for nodes from the `{#node}` root-level object property are listed below.

Table

Object property	Description	Type	Example
id	Node ID	string	975de338-90ff-41ab-9de3-3890ff41ab62
shardingTags	Node sharding tag	array of string	[internal,external]
tenant	Node tenant	string	Europe
version	Node version	string	3.14.0
zone	Zone the node is grouped in	string	europe-west-2

Example

Get the version of a node: `{#node.version}`

Other tips and tricks

Cast data

To convert or cast a string value into an integer, use the following EL:

```
{T(java.lang.Integer).parseInt("some_string")}
```

Evaluate different field types

Use this technique when comparing values of different field types. For example, if `subscription.metadata['my_key']` is a string but the value in request/message content is numeric, use the following EL to compare them:

```
{(""+#jsonPath(#message.content, '$.customerId')).equals(""+
#subscription.metadata['my_key'])}
```

Compare values

Gravitee reads EL data by looking for expressions starting with `{`, followed by `#`, `T`, or `(`. If it finds one of these patterns, it treats the entire string as a single expression to evaluate. Otherwise, it treats it as a string template.

This won't work:

```
{"PARTNER".equals(#subscription.metadata['clientType'])}
```

These will work:

```
{("PARTNER".equals(#subscription.metadata['clientType']))}
{("PARTNER").equals(#subscription.metadata['clientType'])}
{#subscription.metadata['clientType'].equals("PARTNER")}
```

`{#request.content}` **is only available for policies bound to an** `on-request-content` **phase.**